YouSpeed Technical Report: System Architecture, Jurisdiction Handling, Bundle Generation, and Lightweight Matching Benchmarks for Offline Smartphone Speed-Limit Assistance

Raphael Volz
Pforzheim University
raphael.volz@hs-pforzheim.de

March 17, 2026

Abstract

This technical report documents the current YouSpeed system as an offline-first deployment stack for smartphone speed-limit assistance. Its central claim is deliberately narrow: the report identifies which parts of the present system have converged into stable implementation decisions, and which parts remain open matcher-design trade-offs. The stable layer is the local-first bundle architecture: regional OpenStreetMap-derived artifacts, on-device area-context lookup, coverage-aware bundle routing, jurisdiction-specific warning rules, and a publication pipeline that already supports regional sharding and iterative delta updates across the iPhone release-track client and the Android internal alpha. To explain why this stack is necessary, the report quantifies speed-limit provenance in Germany and explicit `maxspeed` coverage across 45 European country extracts. It then evaluates two different questions separately. At the data-layer level, four runtime architectures (S1–S4) are compared under a fixed lookup harness; S3, a single-file spatially indexed embedded database, remains the strongest present runtime choice. At the matcher level, an all-log causal complexity ladder is used only as a diagnostic screen for the shipped matcher family, while closed-loop replay first compares five deployable profiles and then extends the lightweight branch through a later 10-model sweep on the full fixID-carrying log inventory. These replay experiments narrow the model-selection conclusion substantially. The simpler M2 nearest-plus-street-ref matcher now delivers practically the same aggregate replay quality as the richer-data variants: the strongest later profiles improve aggregate accuracy by only 0.30 percentage points, and direct `way_links` ablations leave M2's primary replay metrics unchanged while shrinking the bundle materially. The report therefore argues for a settled deployment architecture together with a narrower remaining frontier: targeted refinements on top of a lightweight matcher rather than ever richer side-table machinery.

Keywords: intelligent speed assistance offline-first OpenStreetMap map matching mobile GIS smartphone systems

1 Introduction

Intelligent speed assistance (ISA) is now a regulatory requirement for new vehicle types in the European Union, yet the installed vehicle base remains far larger than annual new registrations [1; 2; 3; 4; 5]. This keeps the retrofit question open: practical speed-limit assistance still needs to run on devices that drivers already carry. Smartphone deployment is therefore relevant not as a theoretical alternative to integrated vehicle systems, but as a current path to broad availability.

YouSpeed addresses that retrofit path with a local runtime that keeps map data, area-context lookup, matching, and local correction handling on device. The current codebase spans a shared preprocessing and bundle-publication pipeline, an iPhone client on the launch path, and an Android internal alpha that follows the same bundle contract and the same deployed `corridorHMM` matcher logic. At this stage, the important question is no longer whether offline inference is possible in principle. The practical question is which parts of that stack have already converged into stable deployment choices and which parts are still being optimized under product constraints.

This report treats those two decision types separately. The first is the platform decision: how map data is packaged, routed, updated, and combined with jurisdiction-aware speed inference on device. The second is the matcher decision: which bounded local profile should run on top of that platform, and how much sequence machinery is worth carrying in the current consumer apps. Jurisdiction handling and bundle publication matter in this storyline because they explain why the runtime cannot be reduced to nearest-road lookup on a static country file.

The paper therefore proceeds in one direction rather than several competing ones. Sections 2–4 document the implemented system, the provenance structure of the speed-limit data it consumes, and the bundle/update pipeline that feeds the mobile clients. Sections 5–6 then evaluate the two remaining decision layers separately: a runtime-data architecture benchmark and a replay-based matcher comparison. The core result is intentionally asymmetric. The local-first delivery stack and the S3 data layer are now stable. The expanded replay sweep also weakens the case for richer matcher data structures: M2 already operates in the same practical quality band as the later topology- and corridor-heavier variants. The remaining frontier is therefore targeted lightweight refinement rather than architecture selection or ever richer bundle-side state.

2 Related Work

Map matching has long been framed as a trade-off between geometric simplicity and sequence-aware robustness. The classic survey by Quddus et al. [6] organizes the field into geometric, topological, and probabilistic families. Sequence models then become the dominant answer to noisy and sparse trajectories: the HMM formulation of Newson and Krumm [7], ST-matching by Lou et al. [8], online HMM variants for streaming data [9; 10], low-latency path inference by Hunter et al. [11], and faster precomputed transition schemes such as FMM [12]. These methods demonstrate that later context is powerful, but they also assume a route-inference budget that is larger than the one targeted by the current YouSpeed runtime.

Smartphone-specific work confirms that this literature transfers to consumer sensing, but again usually in a richer probabilistic setting than the one used here. Bierlaire et al. [13] evaluate a probabilistic matcher directly on smartphone GPS traces, and production engines such as OSRM, Valhalla Meili, and Mapbox expose candidate radiuses, transition controls, and confidence-style outputs as first-class API concepts [14; 15; 16]. The present report borrows the scientific lesson that sequence context matters, while keeping the implementation target smaller and fully local.

Recent learning-based work pushes map matching further toward transfer learning, personalization, and adaptation under limited labels. Transformer transfer learning [17], deep inverse reinforcement learning with multi-intention modeling [18], preference-aware HMMs [19], and meta-learning approaches [20] all show that richer models can exploit driver- or context-specific structure. These lines are important context for YouSpeed, but they sit one layer above the current report. The present system still prioritizes bounded local state, auditable heuristics, and direct replay evaluation on device-shaped corpora.

The report also sits close to deployment-oriented packaging work rather than only to inference models. Single-archive spatial delivery formats such as MBTiles and PMTiles show that packaging choices are themselves a mature systems topic [21; 22]. That point matters because YouSpeed currently compares not only matching logic but also how map data is physically packaged, updated, and routed at country scale. The distinctive contribution of this report is therefore the joint treatment of lightweight matching, jurisdiction handling, mobile-client runtime scope, runtime data architecture, and bundle publication under present product constraints.

3 System Architecture

3.1 Current end-to-end architecture

The implemented YouSpeed path is local-first and offline-first. OSM snapshot ingestion and preprocessing run off device. The build pipeline emits regional v3 bundles that package road geometries, speed tags, area-context polygons, coverage metadata, and optional corridor tables in a single runtime artifact. Mobile clients load bundled target manifests, choose country or regional artifacts, verify checksums, and activate

bundles atomically. During driving, each fix is routed to the best downloaded bundle through a coverage bounding-box prefilter followed by coverage-polygon containment. The selected bundle then executes local lookup, speed derivation, and way selection without a route-level cloud dependency.

Figure 1 summarizes the current paper-scope architecture. The broader cloud corroboration stack that appears in internal architecture documents remains a north-star extension, not the present critical path. The evaluated critical path is snapshot-to-bundle publication, local bundle activation, on-device inference, and editor-mediated export of reviewed local observations.

The benchmark harness sits beside this runtime rather than replacing it. The S1–S4 latency benchmark reuses the same Germany-derived artifacts and query primitives but isolates spatial lookup and area-context retrieval at one fixed probe point. The replay benchmarks reuse the same runtime matcher implementations and logs, but they remove the user-interface lifecycle so that candidate arbitration, tunnel handling, and oscillation behavior can be measured directly.

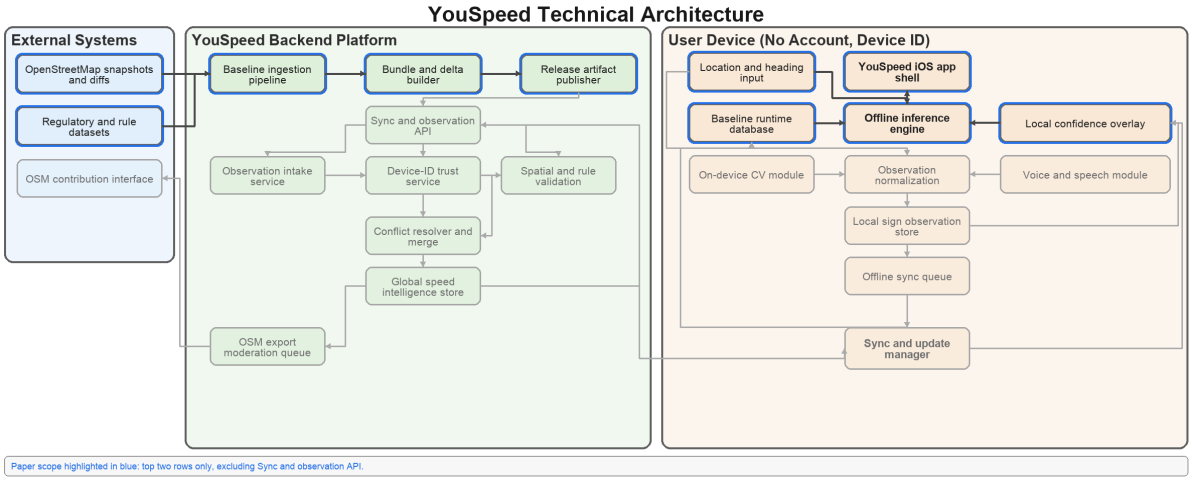

Figure 1: Current paper-scope system architecture. The implemented critical path is local-first: regional bundle publication, on-device inference, and editor-mediated observation export.

3.2 OpenStreetMap data model and runtime normalization

OpenStreetMap is not stored as one road-centric table. Its primary objects are nodes, ways, and relations, while semantics are expressed by tags rather than by a fixed relational schema. For YouSpeed, that collaborative data model has direct runtime consequences. Car-drivable road segments are represented as tagged ways; speed information may appear as an explicit numeric `maxspeed=*`, as inherited rule tokens such as `maxspeed:type` or `source:maxspeed`, or be absent altogether [23; 24]. Built-up context, city naming, and boundary semantics live on separate polygonal objects. One logical street is often split into many ways, and the same street-reference token can recur across nearby carriageways, ramps, or service roads. Daily diffs therefore move not only speed tags but also geometry, connectivity, and area context.

That flexibility is valuable for open collaborative mapping, but it is not the same shape as the low-latency per-fix lookup problem solved by the mobile clients. The app therefore does not consume raw OSM elements directly. The bundle builder normalizes them into runtime tables whose contents match the questions asked during driving: candidate-way retrieval, geometry refinement, built-up inference, continuity gating, corridor state, bundle routing, and jurisdiction-rule selection. Table 1 makes that translation explicit and shows why data generation and app inference are one connected system rather than separate topics.

3.3 iPhone application

The iPhone application is the current release-track client. It bootstraps from a bundled Karlsruhe seed, discovers Germany shard manifests from the bundled `BundleTargets.top10.json` contract, downloads full bundles or verified SQL delta chains when available, assembles multipart database payloads

Table 1: How the collaborative OSM data model is normalized into the mobile runtime.

OSM-side structure	Runtime normalization	On-device use
Car-drivable ways with `highway`, `maxspeed`, `maxspeed:type`, `source:maxspeed`, `ref`, tunnel, and service tags	`ways`, `way_geom`, `ways_rtree`	Candidate generation, geometry refinement, and explicit or inherited speed derivation.
Residential polygons, place or admin context, and other area features	`areas`, `areas_rtree`	Built-up inference, city labels, and locality-sensitive warning context.
Shared endpoints, repeated refs, and continuity clues distributed across adjacent ways	`way_links`	Connected-transition gating and continuity support across way boundaries.
Tunnel and motorway corridor fragments spread across many tagged ways	`corridor_progress`, `corridor_pairs`	Portal-aware tunnel or motorway sequence reasoning.
Country identity, coverage polygons, and release metadata	Bundle manifests and country catalogs	Bundle routing, rule selection, checksum validation, and update eligibility.

when needed, and activates bundles atomically. The runtime uses plain `SQLite3` rather than runtime `mod_spatialite` loading, with an RTree bounding-box prefilter, polyline distance refinement, optional heading-aware scoring, deterministic tunnel handling, and bundle-coverage routing across downloaded regions.

The same client also carries the current driver-facing feature surface. It presents current speed, inferred limit, street and city context, tunnel state, and penalty-aware overspeed warnings. It captures local speed-limit observations through an on-device speech-recognition workflow, stores them in a review queue, and exports approved proposals as editor-oriented `.osc` packages rather than publishing directly from the app. Figure 2 shows representative driver-facing states from the web-facing screenshot set used by the product site. The iPhone codebase is also the main source of inspector logs and replay corpora used in the matching benchmarks reported later in this report.

3.4 Android application

The Android application is an internal alpha that follows the same bundle and manifest contract as the iPhone client, but it is not the current public release gate. It bootstraps from a bundled seed, derives the same Germany shard endpoints, verifies and activates full bundles or multipart bundle payloads, and mirrors the same local lookup contract. The Android matcher now executes the same deployed `corridorHMM` path as the iPhone runtime: geometry-first candidate ranking, corridor prefiltering, connected-transition pruning over `way_links`, continuity arbitration, short-history mini-HMM re-ranking, a three-way expert gate, and final tunnel or corridor locks backed by `corridor_progress` and `corridor_pairs`. It also includes a fallback path for Android builds that expose stored RTree tables without exposing the SQLite RTree module at runtime.

The Android alpha emphasizes parity experiments that are useful for architecture validation. It adds spoken overspeed and driving-ban warnings, a bundled German offline Vosk recognizer for local speed capture, SQLite-backed review and export of local observations, debug log export, OSM browser handoff, and emulator or device instrumentation for replay and live bundle bootstrap. This makes Android scientifically relevant even before public launch because it exercises the shared bundle contract and runtime logic under a second mobile platform.

3.5 Execution domains: generation, live inference, and a posteriori analysis

The report spans three execution domains that should not be conflated. Bundle generation and publication are off-device preparation tasks. Live inference is the driver-facing path executed on the phone. A posteriori log replay is an offline analysis path that consumes inspector logs after the trip. Table 2 separates these domains because the evaluation later in the paper depends on using future information offline without implying that the same information is available during live assistance.

Figure 3 shows the browser-based inspector used for this off-device replay-analysis path: per-fix summaries,

Figure 2: Current iPhone warning surface as shown in the web-facing product screenshots. Left: penalty-aware overspeed state with driving-ban warning. Right: pedestrian-zone low-speed presentation.

Table 2: Execution domains covered by the report.

Domain	Where it runs	Main inputs	Main outputs
Bundle generation and publication	Off device	OSM snapshots or diffs, target-country config, sharding limits, rule assets	SQLite bundles, manifests, country catalogs, delta packs, update indexes.
Live inference and warnings	On device	Active bundle, coverage metadata, GPS fix, short match context, `country_code`, penalty JSON	Matched way, inferred speed limit, street or city context, tunnel state, warning state.
A posteriori log analysis	Off device	Inspector logs with candidate traces, selection traces, and replay corpora	Hindsight pseudo-labels, replay metrics, gate audits, offline training sets.

selection traces, candidate details, and the linked map view are all visible during post-drive review.

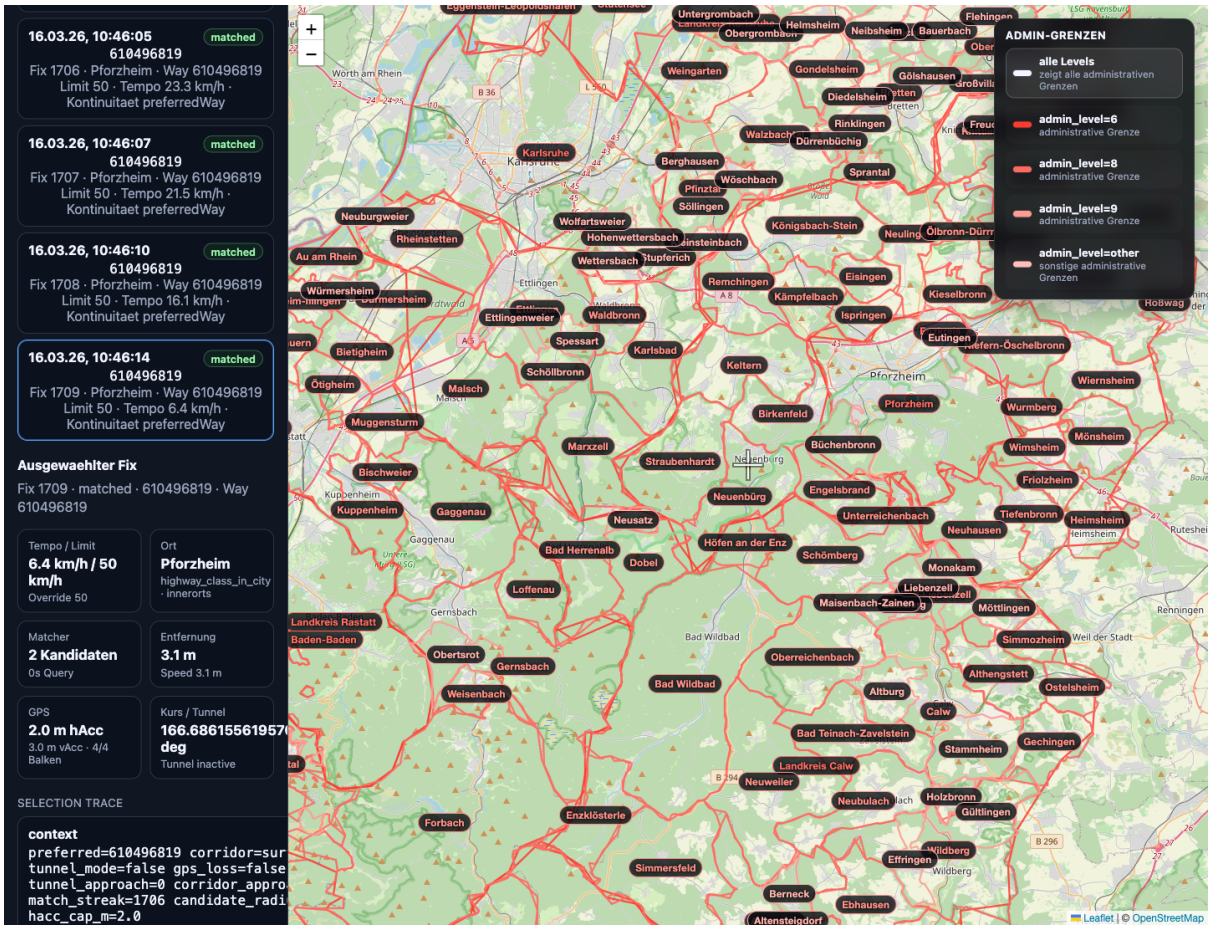

Figure 3: Inspector web app used for replay analysis and per-fix matcher debugging. The view combines fix summaries, selection traces, candidate details, and the linked map context.

3.6 Current consumer-app matcher

The current consumer applications do not run the full seven-stage complexity ladder reported later in the offline benchmark. Both consumer apps now execute the same deployed matcher path: the corridor-aware `corridorHMM` configuration. The smaller `connectedBaseline` branch remains available inside the iPhone service as a benchmark and regression reference, and it also remains one of the runtime profiles evaluated later in this report, but it is not the default consumer runtime on either platform.

The deployed matcher is local and bounded rather than route-global. Each fix is resolved from a local candidate window, but the decision carries short history through a `WayMatchContext` that includes preferred way identity, recent way and street-reference history, recent matched fixes, tunnel approach counters, recent mini-HMM hypotheses, tunnel mode, motorway mode, and active or approaching corridor state. The runtime also consumes three topology side tables from the active bundle. `way_links` stores shared-node reachability and shared-reference relations. `corridor_progress` stores corridor identity together with entry depth, remaining span, and node counts for tunnel and motorway segments. `corridor_pairs` links paired corridor components so that entry and exit chains can be scored consistently across repeated fixes.

The raw mini-HMM stage in Table 17 is the candidate selected at the runtime's logged `mini_hmm` step, before the three-way expert gate and before the later corridor or tunnel hold logic. At fix t, the runtime first keeps the top $B = 4$ candidates after the earlier query, corridor-feasibility, road-graph, and continuity steps. Each beam candidate $i \in C_t^{(B)}$ is then expanded into a stateful hypothesis $h_t = (i, z_t)$, where the

Table 3: Ordered stages of the deployed `corridorHMM` matcher in both consumer apps.

Stage	Role in the runtime	Main evidence or state source
Candidate query	Query nearby ways, refine by polyline distance, optionally add heading mismatch, and keep trace features for later gates.	Local candidate set C_t, distance $d_{t,i}$, heading mismatch $\Delta\theta_{t,i}$, endpoint proximity.
Corridor feasibility gate	Reject candidates incompatible with active or approaching tunnel or motorway corridors, and suppress ambiguous surface-to-tunnel snaps unless recent GPS loss justifies fallback.	`corridor_progress`, `corridor_pairs`, GPS-loss flag, horizontal-accuracy buffer.
Connected road-graph gate	After warmup, remove disconnected transitions and keep only candidates reachable from the recent match state when graph evidence exists.	`way_links`, preferred way, matched-fix count.
Continuity heuristics	Form preferred-way, same-ref, linked-way, and recent-way classes; then apply bounce suppression, transition promotion, low-speed turn overrides, and conservative continuity holds.	Recent way or ref history, endpoint proximity, speed, heading, graph links.
Mini-HMM beam	Score a short beam of candidates together with tunnel or motorway sequence states, using emission penalties plus decayed transition costs from recent hypotheses. The raw mini-HMM stage in Table 17 is the winner of this step before later gates.	Recent hypotheses, corridor state machine, signal evidence, pair-aware transition penalties.
Three-way expert gate	When the sequence state is still surface-like, arbitrate between current, lowest-distance, and lowest-endpoint experts; veto same-ref bounce and infeasible turns.	Top-2 score margin, speed, accuracy, signal bars, trace ranks, continuity bands, endpoint features.
Final corridor or tunnel gates	Apply `corridor_mode_lock`, `corridor_entry_arm`, `corridor_entry_gate`, `tunnel_entry_gate`, and `tunnel_exit_gate`, then emit the new active or approach corridor state.	Corridor depth or span, entry-zone counters, portal exposure, signal degradation, current tunnel mode.

sequence state z_t is constrained by the candidate class:

$$z_t \in \begin{cases} \{\text{surface}, \text{tunnelExit}\}, & \text{surface way} \\ \{\text{tunnelPortal}, \text{tunnelInside}\}, & \text{tunnel way} \\ \{\text{motorwayPortal}, \text{motorwayExit}\}, & \text{motorway\_link way} \\ \{\text{motorwayInside}\}, & \text{motorway way.} \end{cases}$$

The mini-HMM is not always active. It turns on only when the current fix is ambiguous, when the best geometric candidate conflicts with strong continuity evidence, when a same-ref transition candidate is available, when tunnel candidates enter the beam, or when recent hypotheses already exist from previous fixes.

For each expanded hypothesis, the runtime computes a cumulative cost

$$J(h_t) = e(h_t) + \min_{h_{t-1}} \left[\gamma J(h_{t-1}) + \tau(h_{t-1}, h_t)\right],$$

where $\gamma = 0.55$ is the history decay and the initial step reduces to $J(h_t) = e(h_t)$ when no recent hypotheses exist. The emission term is

$$e(h_t) = s_{\text{geom}}(t, i) + p_{\text{prior}}(t, i) + p_{\text{state}}(t, i, z_t).$$

Here $s_{\text{geom}}(t, i)$ is the heading-aware geometric score already computed at candidate-query time. The prior term p_{prior} encodes short-history bias from the current context: the preferred way receives a -8.0 reward, recent ways -2.5, recent or preferred street-reference agreement -3.5, tunnel candidates receive a further -6.0 reward when the runtime is already in tunnel mode, non-tunnel candidates receive a $+10.0$ penalty in that same case, and recent tunnel exposure under GPS loss adds an extra -2.0 reward. The state-emission term p_{state} scores whether the candidate is compatible with the hypothesized sequence state. Illegal state-candidate combinations incur a large $+80.0$ penalty; plausible transitions use a small expected-transition cost of $+2.0$ or persistence cost of $+0.5$; tunnel and motorway evidence then subtract rewards based on portal progress, signal degradation, corridor-chain depth, and paired corridor relations from `corridor_pairs`. In practice, this term is what allows the raw mini-HMM stage

to favor tunnel-inside or motorway-inside trajectories even when a single-fix surface candidate remains geometrically attractive.

The transition term decomposes as

$$\tau(h_{t-1}, h_t) = \tau_{\text{way}}(h_{t-1}, h_t) + \tau_{\text{state}}(h_{t-1}, h_t).$$

The way-transition part τ_{way} is zero for staying on the same way, 0.75 for shared-reference linked transitions, 2.0 for endpoint continuations, 2.5 for graph-linked transitions, 4.0 for same-reference jumps without an explicit graph link, 8.0 for recent-way revisits, 12.0 for same-highway-class fallback transitions, and 18.0 for unrelated transitions. This base cost is then increased by same-reference bounce suppression and heading-feasibility penalties when the implied turn is not kinematically plausible. The state-transition part τ_{state} carries the tunnel and motorway sequence machine. It rewards legal persistence inside committed tunnel and motorway states, penalizes re-entry, forbids incompatible cross-state jumps, and uses paired-corridor relations so that entry and exit chains on opposite sides of a tunnel or motorway corridor can be connected consistently over repeated fixes.

After all candidate-state hypotheses are scored, the runtime sorts them by cumulative cost, keeps the best four as the next `recentHypotheses`, and emits the lowest-cost candidate as the raw mini-HMM winner. The later runtime is allowed to override that winner only in specific ways: committed corridor promotion, the three-way expert gate, same-ref bounce suppression, turn-feasibility vetoes, and the final corridor or tunnel hold logic. The subsequent three-way gate is narrower: it activates only when the runtime is not already committed to a tunnel-inside or motorway-inside state, and it chooses among the current selection, the lowest-distance expert, and the lowest-endpoint expert using a compact learned gate over trace features rather than a full end-to-end neural matcher.

The on-device inference contract extends beyond way selection. Once a way is selected, the mobile runtime derives the effective speed limit from explicit or inherited speed tags, highway class, area context, and the bundle's `country_code`; it then applies country-specific penalty configuration to produce the warning surface shown to the driver. This live path uses only the active bundle, recent match context, and bundled rule assets. The hindsight pseudo-labels, replay corpora, and gate audits discussed later are strictly offline analysis tools and are never consulted by the consumer runtime during driving. The current consumer apps also do not expose a calibrated posterior match probability to the driver; instead, they log candidate scores and gate activations so that match confidence can be estimated retrospectively from replay stability.

Table 4: Current matcher deployment and why Android parity converges iteratively.

Factor	Evidence in the current repository	Consequence for parity work
Platform lead	`V3SpeedLimitService.swift` enters the repository on 2026-02-23 and receives 14 commits through 2026-03-13, while `V3SpeedLimitLookup.kt` enters on 2026-03-13 and currently has two commits.	Android ports a moving implementation rather than co-evolving from the first matcher revision.
Experiment asymmetry	Most closed-loop matcher experimentation remains iPhone-led through `SpeedConsumerTests.swift` (8,330 lines, 19 commits), `SpeedDBBenchSketch`, and the current matcher-metrics collection script. Android replay coverage exists, but its matcher-specific instrumented file is newer and smaller.	New matching behavior is observed, tuned, and accepted on iPhone first, then carried to Android.
State-space coupling	Exact parity depends on `WayMatchContext`, recent hypotheses, tunnel-approach accumulation, `way_links`, `corridor_progress`, `corridor_pairs`, and late gates such as `corridor_mode_lock` and `tunnel_entry_gate`.	Small omissions leave single-fix behavior apparently close while multi-fix replay behavior still diverges.
Duplicated matcher cores	The production matcher exists as separate Swift and Kotlin implementations rather than as one shared generated core.	Semantic drift accumulates whenever one platform changes first and the second platform must re-port the same state machine.
Replay-only failure modes	Several gates activate only after repeated portal exposure, degraded signal quality, entry-zone depth accumulation, or tunnel-mode carryover.	Feature parity requires closed-loop replay and device instrumentation, not only isolated geometric fixtures.

Android now implements the same ordered matcher stages as iPhone. The remaining cross-platform differences are outside the matcher core: release status, speech stack, user-interface details, and Android's

runtime fallback when stored RTree tables exist but the SQLite RTree module is not exposed by the device build.

Table 5: Current mobile-client feature scope.

Capability	iPhone release-track app	Android internal alpha
Bundle lifecycle	Bundled seed, Germany shard discovery, full-bundle or SQL-delta updates, multipart assembly, atomic activation.	Bundled seed, shared shard discovery, full-bundle bootstrap, multipart assembly, atomic activation.
Lookup and matching	S3 `SQLite3` lookup with the app-default `corridorHMM` matcher, connected-baseline branch retained for benchmarks, coverage-based regional routing, deterministic tunnel and corridor handling.	Shared v3 lookup contract with the same deployed `corridorHMM` matcher, the same corridor-pair and tunnel gates, and bbox fallback when Android lacks live RTree support.
Warning surface	Driver-facing speed or limit UI, tunnel state, penalty-aware overspeed presentation, haptic and speech driving-ban alerts.	Compose parity UI, spoken overspeed and driving-ban alerts, tunnel and matcher debug state.
Local observations	On-device speech capture, review queue, single or bulk `.osc` editor export.	Bundled Vosk offline speech capture, SQLite review queue, single or bulk `.osc` export, OSM browser handoff.
Verification	Unit tests, route replay, tunnel regressions, geometry-stress analysis on iPhone logs.	Emulator or device instrumentation, live Germany-shard bootstrap test, replay regressions, runtime fallback checks.

4 Jurisdiction Handling and Maxspeed Provenance

4.1 Separation of bundle identity, speed inference, and warning semantics

The current system handles jurisdiction-specific behavior in layers rather than through one monolithic legal model. At bundle level, the manifest can carry a three-letter `country_code`, optional coverage metadata, and an optional country-specific penalty-rule artifact. At inference level, the way table stores explicit `maxspeed`, inherited `maxspeed:type`, and inherited `source:maxspeed` strings alongside road class and area-context inputs. At consumer-warning level, the app loads country-specific penalty-rule JSON that can encode different currencies, languages, locality variants, penalty points, and conditional driving-ban logic. This separation makes the runtime robust to incomplete map tagging, but it also means that jurisdiction handling is broader in the warning layer than in the unsigned-road legal-default layer.

Table 6: Current jurisdiction-handling layers in the runtime.

Layer	Mechanism	Current scope and limitation
Bundle identity	Manifest `country_code`, coverage bbox or polygon, country bundle catalog.	Multi-country and multi-region routing is supported in the artifact contract.
Speed-tag ingestion	`maxspeed`, `maxspeed:type`, `source:maxspeed`, road class, tunnel tag, service tag.	Works wherever explicit or inherited tags exist.
Locality split	`insideCity` derived from way-class precedence first, then residential polygons, then city-context fallback.	Supports inner/outer locality warnings, but the legal-default use is strongest for Germany.
Warning semantics	Country-specific penalty JSON with currency, language, bands, locality variants, and conditional driving-ban metadata.	Present in both mobile apps and broader than the current legal-default inference layer.
Unsigned-road defaults	Inherited `urban`/`rural`/`motorway` parsing, then road-class heuristics with residential override.	Implemented locally, but current semantics are still validated mainly for Germany.

Two current examples show how this split works in practice. The German ruleset encodes separate inner-city and outer-city penalty variants and conditional driving bans for repeat offenses. The Netherlands ruleset uses simpler national bands without a locality split, but it still carries jurisdiction-specific currency, source metadata, and high-overspeed conditions. The runtime resolves these rule differences after overspeed estimation through `country_code` and `insideCity`, not by changing the map-matching core itself.

The currently packaged rule portfolio spans ten jurisdictions: Belgium, Germany, Great Britain, Iceland, Liechtenstein, Luxembourg, Monaco, the Netherlands, Romania, and Sweden. This means that jurisdiction-specific warning semantics already cover a broader set of countries than the present country-specific legal-default inference layer.

The current speed-derivation logic is more conservative. Explicit numeric `maxspeed=*` tags take precedence. If they are absent, the runtime checks inherited tags for strings containing `urban`, `rural`, or `motorway`. If neither explicit nor inherited tags are decisive, it falls back to road-class heuristics and then optionally overrides those heuristics with the current `insideCity` decision. This logic keeps the code auditable and fast, but it is not yet a full country-by-country legal-default engine.

4.2 Germany provenance decomposition

All evaluated artifacts derive from the same Germany OSM snapshot (`germany-latest.osm.pbf`, 4.70 GB) [25]. Table 7 summarizes the current Germany dataset profile and the provenance decomposition that the runtime sees after rule-aware fusion.

Table 7: Germany dataset profile and speed-limit provenance used by the current runtime.

Metric	Value
Input file size	4.70 GB
Car-drivable ways	7,963,481
Ways with `maxspeed=*`	2,637,020
Ways with `zone:maxspeed=*`	172,116
Ways with explicit speed-limit tag	2,639,693
Area-context objects	129,459
Explicit tagged limit share	33.15%
Implicit regulatory tag share	0.98%
Context-based legal fallback share	65.87%

The Germany profile is the central reason the report keeps provenance explicit. Roughly one third of car-drivable ways already carry explicit speed limits in OSM. Another 0.98% contribute inherited regulatory context through rule-bearing tags. The remaining 65.87% depend on default interpretation or area context. That asymmetry is exactly why the product has to couple map matching with area-context lookup and why pure explicit-tag retrieval would be insufficient even before matching errors are considered.

The current area-context logic is also deliberately narrower than administrative-jurisdiction modeling. For limit inference, the runtime first uses way-class precedence (`residential`, `service`, `crossing`, `living_street`) to assert built-up context where appropriate. If that does not decide the case, it resolves residential polygons. Only then does it use broader city context as a fallback signal. This means that `insideCity` in the current runtime is a built-up-driving proxy, not a general administrative-boundary classifier.

4.3 Cross-country explicit-tag statistics

The repository also contains a Europe-wide comparative statistic over 45 country extracts. This analysis counts car-drivable ways and explicit `maxspeed=*` coverage per country, using the same drivable-highway definition as the bundle generator. It is therefore a cross-country provenance statistic for explicit-tag availability, not a full country-by-country decomposition into explicit, inherited, and fallback classes.

The heterogeneity is substantial. The explicit-share range spans from 3.10% (Turkey) to 67.75% (Netherlands), with a median of only 16.15%. Germany sits above both the European median and the European weighted mean, but it is still far from the top of the distribution. This matters for system design: a country portfolio chosen purely by map size would not necessarily align with a country portfolio chosen by explicit-tag completeness.

The current bundled target set is therefore not arbitrary. Its mean explicit-tag share is 42.97%, compared with 20.83% across all 45 European extracts, and its way-weighted mean is 39.25%. In other words, the current target set clusters well above the continental baseline for explicit-tag availability. This does not remove the need for fallback logic, but it does reduce the probability that the initial rollout set is dominated by very low-explicitness jurisdictions.

Table 8: Descriptive statistics for explicit `maxspeed=*` share across 45 European extracts.

Statistic	Value
Countries analyzed	45
Minimum country share	3.10%
First quartile	13.11%
Median	16.15%
Arithmetic mean	20.83%
Way-weighted mean	22.61%
Third quartile	25.35%
Maximum country share	67.75%
Germany rank in Europe-wide ordering	8
Germany explicit-tag share	33.12%

Table 9: Current bundle-target countries and their explicit `maxspeed=*` coverage.

Rank	Country	ISO2	Share	Bundle mode
1	Netherlands	NL	67.75%	single country
2	Romania	RO	62.55%	single country
3	Luxembourg	LU	53.60%	single country
4	Belgium	BE	37.06%	single country
5	Sweden	SE	35.89%	single country
6	Liechtenstein	LI	33.96%	single country
7	Iceland	IS	33.78%	single country
8	Germany	DE	33.12%	regional shards
9	Monaco	MC	29.00%	single country

At the same time, the report should be clear about what this comparative statistic does not yet provide. The Europe-wide analysis currently measures explicit-tag prevalence, and the Germany snapshot already carries a fuller provenance split because its fallback semantics are the best validated in the current codebase. Extending the same explicit or inherited or fallback decomposition to every target country is possible, but it is not yet the standard comparative report generated by the current toolchain.

5 Bundle Generation and Iterative Updates

5.1 Country selection and regional sharding

Country selection is governed by two measurable quantities: explicit-tag coverage and artifact size. The script `analyze_europe_maxspeed_share.py` scans European Geofabrik extracts, counts car-drivable ways, and produces the cross-country explicit-tag ranking discussed above. That ranking feeds the bundle-target configuration used by the apps. Sharding mode is then determined by country PBF size rather than by manual preference alone.

That split is governed by a simple operational rule: if a country PBF stays at or below a 1 GB threshold, it remains a single bundle; if it exceeds the threshold, the planner switches to Geofabrik child regions. Germany therefore becomes a 16-region shard set in the current target file, while smaller countries such as the Netherlands or Luxembourg remain single-country bundles. This rule does not optimize every possible deployment objective, but it is transparent, reproducible, and aligned with mobile download constraints.

5.2 SQLite bundle construction

The current S3 runtime artifact is a normalized SQLite database constructed by `build_spatialite_v3.py`. It materializes the OSM-derived runtime view needed by the clients: car-drivable ways with speed-relevant attributes, RTree bounding boxes for fast pruning, polyline geometries for point-to-segment refinement, and polygonal context for residential inference. Optional topology and corridor stages add the detailed shared-endpoint graph and paired corridor tables consumed by the richer matcher profiles.

The current corridor-aware Karlsruhe bundle illustrates the resulting artifact scale. Its latest research bundle contains 242,359 way rows, 32,872 area rows, 435,372 detailed way-link rows, 11,961 corridor-progress rows, and 1,656 corridor-pair rows. Metadata in the same bundle records the spatial backend (`sqlite_rtree`), the detailed link mode (`shared_endpoint_detailed`), and the corridor mode (`paired_portal_chain_v1`). This is the bundle family that underlies the corridor-aware replay numbers reported later.

5.3 Publication artifacts and delivery contract

Publication exposes that runtime database through a delivery contract rather than as a bare file copy. Table 10 summarizes the current stages.

Table 10: Current bundle-generation and publication stages.

Stage	Main script	Primary output	Role
Comparative scan	`analyze_europe_maxspeed_share.py`	Europe ranking CSV and markdown	Country prioritization by explicit-tag coverage.
Sharding plan	`plan_country_region_bundles.py`	Country bundle-plan JSON	Decide single-country vs regional shards.
Bundle build	`build_spatialite_v3.py`	`speeds_v3.sqlite`	Create current S3 runtime database.
Manifest publish	`publish_v3_bundle.py`	Bundle directory plus manifest	Attach db artifact, optional split parts, coverage, penalty rules, and delta index.
Country catalog	`build_v3_country_bundle_catalog.py`	Country catalog JSON	Aggregate regional manifests for country-level discovery.
Target config	`generate_v3_country_bundles.py`	`BundleTargets.top10.json`	Ship the current mobile download target list.

The bundle manifest is deliberately richer than the minimal local file copy. It can reference one full database, an optional list of split `db_parts`, an optional delta index, an optional penalty-rule JSON, and optional coverage metadata. If a bundle exceeds the release-asset threshold, the publisher splits the database into numbered parts while keeping one logical `db` payload in the manifest for identity and checksum semantics. Country catalogs then aggregate multiple regional manifests and their coverage metadata for bundle routing on device.

5.4 Iterative updates

The iterative-update path is already implemented, even though the current release-track behavior remains conservative. Its inputs are daily Geofabrik replication diffs, its artifacts are exact SQL delta packs plus delta manifests, and its release surface is a rolling index of applicable bundle-version transitions. `build_v3_delta_pack.py` either infers a patch directly from an `.osc(.gz)` diff plus the base database or materializes an exact SQL patch by diffing a rebuilt target database against that base database. `roll_v3_delta_index.py` merges resulting manifests into a rolling index with the current retention window of 30 entries.

This update path is not only specified; it is also measured. The current daily-diff analysis covers a 30-day Germany window from 2026-01-22 to 2026-02-20. Table 11 summarizes the workload seen by the update machinery.

These update metrics refer to different operational burdens. Changed ways per day counts OSM ways touched by the daily diff. Maxspeed-tag changes per day isolates direct edits to speed-relevant tags. Invalidated S1 and S2 tiles count partitions that would require republishing under the corresponding tiled layouts. V3 and V4 SQL patch rows count row-level database updates emitted into exact delta packs. Patch apply time measures the host-side execution time of those SQL patches against the base database. The table therefore characterizes update maintenance cost rather than lookup quality.

Table 11: Germany daily-diff update statistics over the current 30-day analysis window.

Metric	Mean	Median	Max
Changed ways per day	60,972	61,076	69,793
Maxspeed-tag changes per day	4,667	4,643	5,668
Invalidated S1 tiles	6,712	6,728	7,756
Invalidated S2 tiles	2,685	2,684	2,907
V3 SQL patch rows	134,965	131,253	158,028
V3 patch apply time (ms)	1,024	1,026	1,180
V4 SQL patch rows	182,929	177,938	213,766
V4 patch apply time (ms)	1,357	1,359	1,581

Three observations follow from these data. First, day-to-day map maintenance is not sparse at country scale: the median day still moves more than 61,000 ways and more than 4,600 speed-related tags. Second, the update burden is architecture-dependent. The same diff window invalidates roughly 2.5 times as many S1 tiles as S2 tiles, which is one reason why coarse global indexing remains unattractive for mobile delivery. Third, exact SQL patch application is already operationally plausible for S3-sized artifacts in the current analysis, with a median simulated apply time of about 1.03 s. This does not yet prove that every mobile update path is production-ready, but it does move iterative updates from abstract future work into an implemented and measured subsystem.

6 Data and Methods

The evaluation is split into two evidence layers because they answer different questions. The S1–S4 benchmark is a storage-and-query microbenchmark over a fixed probe and does not attempt to reproduce the full recurrent app runtime. The matcher study then reuses the real runtime implementations and replay corpora to compare bounded local profiles under closed-loop conditions. Read together, these layers support deployment decisions at different levels: data architecture first, matcher profile second.

Table 12: Approach families evaluated in the report and the decision each one supports.

Family	What varies	Main evidence layer	Decision supported
S1–S4 runtime architectures	Packaging and spatial access path for the same source data	Fixed-probe lookup latency and update-workload proxies	Data-layer selection.
Causal ladder stages	Incremental logic cuts inside the deployed matcher family	Hindsight-labelled agreement metrics and adjacent net corrections	Which added mechanisms help before full replay integration.
M1–M5 deployable profiles	Live S3 matcher profiles with different side tables and gates	Closed-loop replay quality, service latency, bundle size, and oscillation diagnostics	Which consumer profile is worth shipping or simplifying toward.

6.1 Runtime architecture scenarios

The four runtime architectures are summarized in Table 13. The benchmark keeps source data and scoring fixed so that architectural differences arise from packaging and spatial access, not from different road subsets or different ranking rules.

6.2 Investigated matching approaches

All investigated matchers operate on the same local candidate sets. For each fix t, candidate ways $i \in C_t$ receive a geometry score

$$s_{\text{geom}}(t,i) = \begin{cases} d_{t,i} + \lambda_h \cdot \Delta\theta_{t,i}, & \text{if course is reliable} \\ d_{t,i}, & \text{otherwise} \end{cases}$$

Table 13: Current runtime architecture scenarios evaluated in this report.

Scenario	Runtime artifact	Query characteristics
S1	Global country-scale index files	Broad candidate scans with high IO and weak locality.
S2	Content-addressed spatial tile packs	Localized reads and bounded regional fan-out.
S3	Single-file spatially indexed embedded database	Direct spatial-window retrieval with low deployment overhead.
S4	Embedded database with tile-window pre-filter	Two-stage pruning for dense candidate regions.

where $d_{t,i}$ is point-to-way distance and $\Delta\theta_{t,i}$ is bidirectional heading mismatch. Higher-level approaches then add bounded continuity state, short-history sequence logic, or lightweight learned ranking on top of this shared local candidate generation.

The all-log matcher analysis in this section is not meant to choose a shipping matcher directly. In the current product code, both iPhone and Android run the corridor-aware `corridorHMM` branch as the deployed consumer matcher, while the `connectedBaseline` branch remains a benchmarkable alternative inside the iPhone service and one of the five deployable matcher experiments compared later. The log benchmark is therefore used as a discovery screen: it compares causal stages on fixed candidate windows so that the benefit of each added mechanism is explicit before closed-loop integration work. The actual deployment choice is still made only in the replay-based profile comparison of Table 18.

The matching study uses two complementary experimental settings. The first is an all-log causal complexity ladder. It fixes candidate windows from all currently available inspector logs, including geometry-stress sessions and repeated route recordings, and then evaluates a staged sequence of causal selectors on those same windows. This benchmark isolates candidate arbitration quality and asks whether each added layer of the deployed matcher family contributes net hindsight corrections. The staged ladder is:

- **Distance only:** choose the candidate with minimum point-to-way distance.
- **Distance plus heading:** choose the candidate with minimum heading-aware geometry score.
- **Continuity trace:** add preferred-way, same-ref, linked-way, and recent-way banding to form the trace score.
- **Corridor/tunnel selectability:** filter out candidates rejected by corridor and tunnel feasibility before reapplying trace ranking.
- **Runtime heuristic:** take the logged heuristic winner before sequence arbitration.
- **Raw mini-HMM pick:** take the logged mini-HMM winner before later conservative holds.
- **Current deployed final:** take the current shipped runtime output.

A non-causal fixed-lag smoother is explored separately during offline experimentation, but it is not included in the main complexity ladder because it consumes future fixes and therefore is not a candidate consumer-app profile.

The closed-loop replay benchmark is narrower and more deployment-oriented. It keeps bundle queries, session state, tunnel logic, and runtime size or latency terms, and therefore compares five named deployable S3 matcher experiments on the same data layer:

- **M1 Connected baseline.** This profile uses exact geometry distance, heading-aware scoring, connected-way continuity, and tunnel or motorway entry constraints derived from a minimal shared-endpoint graph. It does not materialize explicit corridor tables.

- **M2 Nearest + street-ref continuity.** This profile uses the smaller non-corridor bundle family and a deliberately narrow causal rule: if speed is below 30 km/h and horizontal accuracy is better than 10 m, it takes the nearest way by distance; at higher speed it prefers the nearest candidate that shares the previous street ref; and if the previous matched state is tunnel while horizontal accuracy is worse than 10 m, it keeps tunnel-only candidates until signal quality recovers. The street-ref tokens are already stored on the way rows, so M2 adds no auxiliary side table beyond the base way records.
- **M3 M2 + connected-candidate gate.** This experiment keeps the M2 rule, but before selection it removes candidates that are not connected to the recent match state when the `way_links` graph gate is active. It isolates the value of reintroducing graph connectivity to suppress disconnected bridge and tunnel competitors in an otherwise lightweight matcher.
- **M4 Corridor raw mini-HMM.** This profile uses the same corridor tables, local candidate generation, and bounded-history sequence state as the deployed corridor matcher, but it stops at the runtime's raw mini-HMM winner before the three-way expert gate and the later corridor or tunnel hold logic. It is therefore a live truncation candidate, not a smaller bundle family.
- **M5 Corridor-aware final.** This profile uses the same geometric core but adds corridor identity, corridor phase, paired entry and exit chains, the runtime three-way gate, and the later corridor or tunnel lock logic. This is the current deployed consumer matcher.

Experiments M1–M3 reuse the same 108.5 MiB non-corridor artifact in the current replay harness, so their measured bundle sizes are directly comparable. M1 and M3 both require `way_links` because they use graph-derived connectivity. M2 consumes only geometry distance, street-ref tokens already stored on `ways`, tunnel tags, and bounded session state. A dedicated M2 build therefore can omit `corridor_progress`, `corridor_pairs`, and `way_links`. M3 intentionally reintroduces the connected-transition gate and therefore still requires `way_links`.

6.3 A posteriori replay labeling and confidence filter

The replay evaluation uses a posteriori supervision rather than manual per-fix annotation. For a logged fix t, let $F_t = (y^{\text{live}}_{t+1}, \ldots, y^{\text{live}}_{t+W})$ denote the next W logged selected ways. A hindsight pseudo-label $\tilde{y}_t$ is accepted only when the future-majority way $m_t = \text{mode}(F_t)$ satisfies

$$\#\{j : y^{\text{live}}_{t+j} = m_t\} \geq \lceil W\rho \rceil, \qquad m_t \in C_t, \qquad \text{run}_t(m_t) \geq L,$$

where C_t is the candidate set already visible at fix t, ρ is the required future-agreement ratio, and $\text{run}_t(m_t)$ is the consecutive future run length starting at $t+1$. In the current benchmark configuration, $W = 5$, $L = 5$, and $\rho = 0.8$. Because $L = W$, accepted examples correspond to future-stable continuations that were already present in the current candidate set. This is not treated as ground truth; it is a confidence filter that retains only those ambiguous fixes for which later trajectory stability makes one currently visible candidate substantially more plausible a posteriori.

The offline logs also retain the contemporaneous ambiguity signals needed to interpret these accepted examples: candidate count, top-2 score margin, selected rank, pseudo-label rank, heuristic versus mini-HMM disagreement, continuity class, tunnel or corridor selectability, speed, horizontal accuracy, and GPS bars. These quantities support gate audits and offline dataset export. They should not be confused with a live calibrated match probability exposed by the app. The present consumer runtime does not emit such a probability during driving; confidence is estimated retrospectively from future stability together with the logged candidate and gate traces.

6.4 Query modes and benchmark harness

All architecture scenarios use the same query modes:

- `bbox`: distance to the clamped point on a candidate bounding box,
- `polyline`: minimum point-to-segment distance over full geometry,

- `hybrid:` bbox pre-ranking followed by polyline refinement of a small top-N set,
- `polycontainment:` residential-polygon area-context lookup for built-up classification.

The architecture benchmark uses a fixed Berlin probe point $(52.5200, 13.4050)$, heading $90°$, and $k = 5$ candidates. It runs in two contexts: host-side execution on a MacBook Pro (Apple M4 Max, 48 GB RAM, macOS 15.5) and on-device execution on an iPhone 14 Pro (iOS 26.2.1). This harness isolates spatial lookup and area-context retrieval. It does not reproduce the full recurrent app runtime and it does not currently benchmark Android device latency.

6.5 Replay corpora and metrics

The current matcher comparison uses three corpora with different purposes:

Table 14: Current evaluation corpora.

Corpus	Purpose	Logs	Fixes	Hindsight labels
All-log complexity ladder	Candidate-level hindsight benchmark over all current inspector logs, including geometry sessions	13	16,006	9,169
Inspector replay corpus	Closed-loop replay of current runtime behavior on the current fixID-carrying inspector-log subset	17	26,772	16,176
Geometry-stress corpus	Dedicated tunnel and same-ref stress replay	2	2,598	1,410

The expanded replay corpus excludes one top-level inspector log that does not carry `fixID`, because the closed-loop harness aligns outcomes per fix.

The all-log complexity ladder contains 9,169 hindsight-labelled examples, including 640 switch-labelled cases. Each example has a logged live selection y_t^{live}, a hindsight pseudo-label y_t, and a stage prediction $\hat{y}_t^{(k)}$. For each row in the ladder, we report replay-style accuracy a, changed-example recall r, and unchanged-example accuracy u:

$$a = \frac{1}{N}\sum_{t=1}^{N} \mathbf{1}\left[\hat{y}_t^{(k)} = y_t\right], \qquad r = \frac{1}{|S|}\sum_{t\in S} \mathbf{1}\left[\hat{y}_t^{(k)} = y_t\right], \qquad u = \frac{1}{|\bar{S}|}\sum_{t\in \bar{S}} \mathbf{1}\left[\hat{y}_t^{(k)} = y_t\right],$$

where $S = \{t : y_t \neq y_t^{\text{live}}\}$ is the set of hindsight switch examples and $\bar{S} = \{t : y_t = y_t^{\text{live}}\}$ is the set of keep-current examples. Accuracy a therefore measures overall agreement with the hindsight label, changed-example recall r measures how often a stage recovers cases where the live runtime should have switched, and unchanged-example accuracy u measures how often a stage correctly preserves the logged live selection when hindsight agrees with it.

The ladder also reports the net number of hindsight corrections relative to the previous stage,

$$\Delta_k = \sum_{t=1}^{N} \mathbf{1}\left[\hat{y}_t^{(k-1)} \neq y_t \wedge \hat{y}_t^{(k)} = y_t\right] - \sum_{t=1}^{N} \mathbf{1}\left[\hat{y}_t^{(k-1)} = y_t \wedge \hat{y}_t^{(k)} \neq y_t\right].$$

A positive Δ_k means that the added mechanism fixes more hindsight errors than it introduces; a negative Δ_k means that the new complexity regresses more cases than it repairs on the current corpus.

For profile comparison, the primary metrics are replay accuracy a, changed-example recall r, and unchanged-example accuracy u. To summarize replay quality without collapsing everything into one number too early, we first use

$$A = 0.5a + 0.25r + 0.25u.$$

Because the current product still cares about bundle size and service latency, we also report the present internal score

$$S = 100\left(0.7A + 0.2\frac{t_{\min}}{t} + 0.1\frac{b_{\min}}{b}\right),$$

where t is service-query $p95$ latency, b is bundle size, and $t_{\min}$, $b_{\min}$ are the best values among compared profiles. We treat S as a product-weighted summary, not as a universal scientific metric.

6.6 Metric definitions

Table 15 defines the metrics used across the report so that the runtime-architecture study, the hindsight ladder, and the deployable profile comparison can be read on one vocabulary rather than as separate local scoreboards.

Table 15: Metric definitions used across the architecture and matcher evaluations.

Metric	Definition	Preferred direction
Lookup latency	Wall-clock time for one benchmark query under a named query mode (`bbox`, `polyline`, `hybrid`, `polycontainment`) at the fixed Berlin probe.	Lower
Service $p95$	95th percentile per-fix lookup time of one deployable matcher profile during replay.	Lower
Bundle size	Bytes of the runtime artifact required by a deployable profile.	Lower
Accuracy a	Share of hindsight-labelled examples where the predicted way equals the pseudo-label.	Higher
Changed recall r	Share of changed examples, i.e. pseudo-label differs from logged live choice, that the evaluated stage or profile resolves to the pseudo-label.	Higher
Unchanged accuracy u	Share of keep-current examples, i.e. pseudo-label equals logged live choice, that the evaluated stage or profile preserves.	Higher
Net Δ	Hindsight corrections minus hindsight regressions relative to the previous causal ladder stage.	Higher
Replay composite A	Weighted replay-quality summary $0.5a + 0.25r + 0.25u$.	Higher
Internal score S	Product-weighted summary combining replay quality A, service $p95$, and bundle size.	Higher
Geom. agree.	Share of geometry-stress replay fixes whose replayed way equals the logged way.	Higher
Way ABA	Count of $A \to B \to A$ oscillations in successive selected way IDs during replay.	Lower
Same-ref ABA	Count of $A \to B \to A$ oscillations where all three fixes still share the same street-reference token. Used as a secondary stability diagnostic.	Lower

7 Results

The results follow the same split. We first resolve the runtime-data question, then use the matcher ladder as a diagnostic screen, and only then compare deployable matcher profiles. This order matters: the ladder is not a shipping decision table, and the replay table is not a verdict on the S1–S4 storage benchmark.

7.1 Runtime architecture comparison

Table 16 reports the joint latency matrix, and Figure 4 shows the same values grouped by scenario.

Table 16: Joint host/device latency matrix at the benchmark coordinate (ms).

Execution	Mode	S1	S2	S3	S4
host	bbox	5451.24	172.21	64.74	166.26
host	hybrid	10152.12	169.25	29.56	64.00
host	polyline	10300.65	170.65	38.88	72.03
host	polycontainment	1.51	1.07	0.94	0.44
device	bbox	4526.75	232.58	43.77	792.13
device	hybrid	2935.52	61.03	24.10	99.57
device	polyline	3122.36	76.20	39.16	78.54
device	polycontainment	3.14	0.81	0.74	2.79

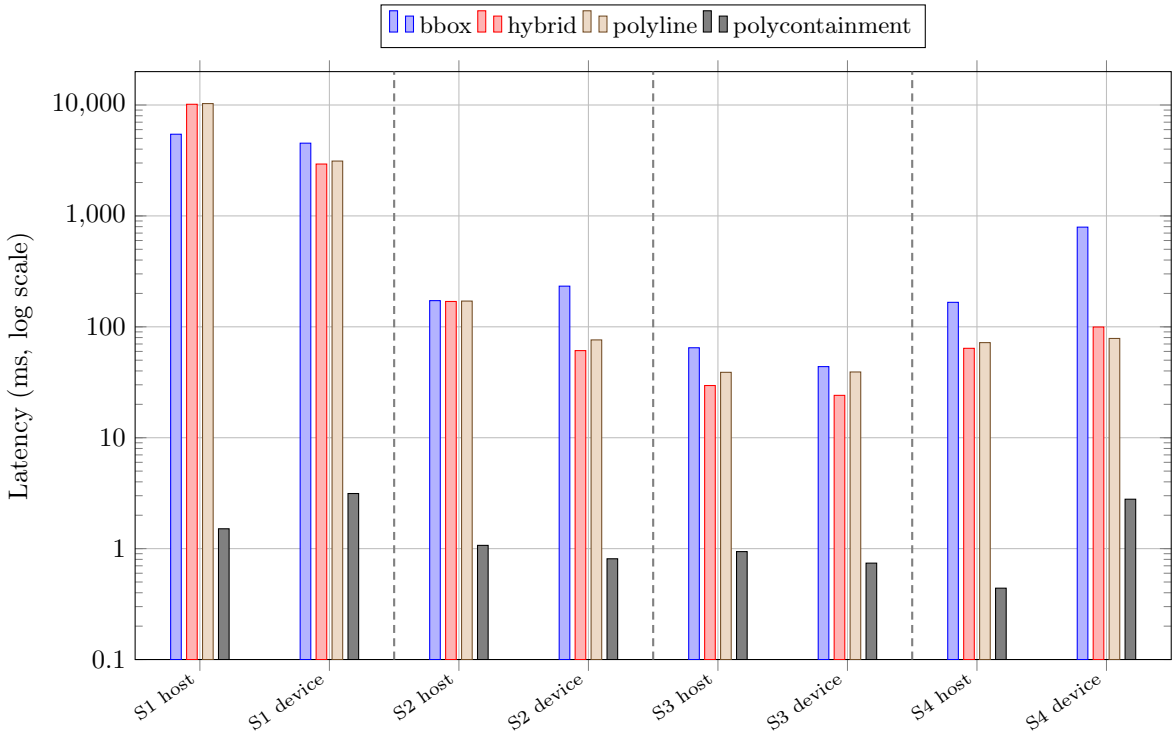

Figure 4: Scenario-first latency view for the current S1–S4 comparison.

The architecture result is clear. S1 is not competitive for country-scale mobile inference. S2 is a substantial improvement and keeps the locality benefits of explicit tiling, but its high file count remains operationally heavier than the single-file variants. S4 does not justify its extra prefilter stage under the current workload; it loses to S3 on device in every decision-relevant query mode. S3 therefore remains the stable data-layer choice in the current project state.

The area-context result is also narrow and current-scope only. The measured `polycontainment` step is a residential-polygon lookup used by the benchmark harness for built-up classification; it is not an administrative-boundary benchmark. Under that definition, S3 remains fastest on device at 0.74 ms.

7.2 All-log complexity ladder of causal matcher stages

Table 17 reports the all-log causal complexity ladder. Its purpose is not to rank heterogeneous offline and online models against each other. Instead, it asks a narrower and more deployable question: on the current full log inventory, what does each added layer of the shipped matcher family contribute beyond the previous causal stage? The final row is the current deployed consumer matcher directly, read from the runtime's logged final trace. The earlier rows are measured intermediate stages from the same local candidate windows.

Table 17: All-log causal complexity ladder for the deployed matcher family across 13 current inspector logs (16,006 fixes, 9,169 hindsight labels). The last column reports net hindsight corrections minus regressions relative to the previous row. The deployed runtime row is marked in bold.

Causal stage	Accuracy	Changed rec.	Keep-current acc.	Net Δ vs. prev.
Distance only	93.98%	57.50%	96.72%	–
Distance + heading	92.44%	43.59%	96.11%	-141
Continuity trace	90.96%	48.91%	94.11%	-136
Corridor/tunnel selectable trace	90.99%	48.91%	94.15%	+3
Runtime heuristic	92.67%	1.09%	99.54%	+154
Raw mini-HMM pick	93.57%	38.44%	97.70%	+82
Current deployed final	93.02%	0.00%	100.00%	-50

Interpretation. The bold row is the exact deployed consumer matcher on this offline corpus: the benchmark

reads the runtime's final selected way from the logged final trace. The preceding rows are not synthetic toy baselines; they are measured causal cut points along the same matcher family. Closed-loop end-to-end evidence for actual deployment choice still appears separately in Table 18.

Four patterns emerge from this ladder. First, added complexity is not monotone. The pure distance baseline already reaches 93.98% accuracy and the strongest switch recovery at 57.50%, which means that many hindsight-labelled switches are geometrically recoverable before any temporal logic is added. Second, the heading-aware geometry score and the continuity-trace layer both move in the wrong aggregate direction on the current all-log corpus: they reduce overall accuracy and incur net regressions relative to the simpler previous stage. Third, the large positive steps come later in the live stack. The runtime heuristic improves stability sharply, raising keep-current accuracy to 99.54% at the cost of almost all switch recovery, and the raw mini-HMM stage then restores much of the lost switch sensitivity while also giving the strongest overall causal operating point in the table at 93.57% accuracy, 38.44% changed-example recall, and 97.70% keep-current accuracy. Fourth, the current deployed final stage is best understood as a conservative hold layer rather than as the strongest hindsight selector: it lifts keep-current accuracy to 100.00%, but it suppresses all 640 hindsight-labelled switches and loses 50 net corrections relative to the raw mini-HMM row. Table 18 then shows why this offline result is not itself a shipping decision: the raw mini-HMM direct live profile does not beat the deployed final profile once closed-loop replay, bundle costs, and runtime side effects are evaluated together.

The all-log ladder is scientifically useful because it separates different notions of improvement before replay integration work begins. It shows that the main benefit of added complexity does not come from the front of the stack. The strongest positive shift occurs when the runtime moves from static trace ranking into explicit bounded-history arbitration, and the strongest overall hindsight stage is the raw mini-HMM rather than the current final conservative hold. This is exactly the role of Table 17: it narrows the candidate space for deployable profile design, not the final shipping decision. In the current report, it successfully nominates one concrete live candidate and the closed-loop benchmark then rejects that candidate.

7.3 Earlier five-profile runtime comparison and updated expanded replay sweep

The earlier five-profile runtime comparison remains useful because it combines bundle size, service latency, and geometry-stress behavior on the narrower original corpus. Table 18 reproduces that baseline. The current model-selection reading, however, comes from a later expanded replay sweep across all 17 fixID-carrying inspector logs (26,772 fixes; 16,176 hindsight labels), summarized in Table 19.

Table 18: Earlier five-profile matcher comparison on the original replay and geometry-stress corpora. The deployed consumer experiment is marked explicitly; the highest earlier internal score S is bolded in the last column.

Exp.	Profile	Bundle (MiB)	Service p95	Replay acc.	Changed rec.	Unchanged acc.	Geom agree.	Way ABA	Score S
M1	Connected baseline	108.5	0.146	92.69%	45.30%	96.24%	80.45%	4	85.79
M2	Nearest + street-ref continuity	108.5	0.140	93.09%	55.17%	95.93%	80.29%	2	**88.46**
M3	M2 + connected-candidate gate	108.5	0.136	92.18%	54.08%	95.03%	81.45%	2	88.35
M4	Corridor raw mini-HMM	128.2	0.152	90.64%	35.27%	94.78%	80.87%	2	80.89
M5	Corridor-aware final (deployed)	128.2	0.151	95.68%	63.17%	98.11%	83.56%	89	88.13

The expanded sweep changes the model-selection conclusion. M2 now provides practically the same aggregate replay quality as the richer-data branches. Relative to the strongest aggregate variants M9 and M10, M2 trails by only 0.30 percentage points of replay accuracy (92.30% vs. 92.60%) and 0.85 percentage points of changed-example recall (30.52% vs. 31.37%), while matching or exceeding the richer variants on tunnel selections (646 vs. 593). M6 is effectively tied with M2, and M7–M10 are best read as incremental logic refinements rather than as evidence that richer bundle-side data is decisive.

Two stronger negative results also emerge. First, M3 again fails to justify reintroducing `way_links`: it matches M2's changed-example recall but loses 1.81 percentage points of replay accuracy and 2.40 percentage points of logged agreement on the broader corpus. Second, the corridor-heavy M4 and M5 profiles are not competitive on this broader corpus. M5 retains the highest switch recall at 49.41%, but it falls to 87.12% replay accuracy and 80.43% logged agreement, which makes it a specialized high-switch profile rather than a generally superior consumer matcher.

Table 19: Expanded replay sweep across 17 fixID-carrying inspector logs (26,772 fixes, 16,176 hindsight labels). Higher is better except tunnel-fix count, which is reported as a diagnostic coverage count rather than as a scalar objective.

Exp.	Profile	Replay acc.	Changed rec.	Log agree.	Tunnel fixes
M1	Connected baseline	90.64%	33.22%	83.03%	550
M2	Nearest + street-ref continuity	92.30%	30.52%	87.46%	646
M3	M2 + connected-candidate gate	90.49%	30.52%	85.06%	642
M4	Corridor raw mini-HMM	82.19%	20.83%	77.45%	481
M5	Corridor-aware final	87.12%	49.41%	80.43%	556
M6	M2 + urban consecutive distance-gap release	92.32%	30.61%	87.46%	646
M7	M6 + 10m search window	92.54%	32.88%	87.33%	619
M8	M6 + no-ref street-name continuity	92.47%	30.86%	87.54%	607
M9	M8 + guarded stale-ref suppression	92.60%	31.37%	87.77%	593
M10	M9 + node-direction-aware junction release	92.60%	31.37%	87.76%	593

Within the later lightweight family, the node-aware endpoint is also less consequential than it first appears. M10 differs from M9 on only one of 26,772 replayed fixes, so even before the bundle ablation there is almost no evidence that additional node-direction logic changes the practical deployment outcome. The most defensible current reading is therefore that M2 already reaches the same practical quality range as the richer-data alternatives, while keeping the cleanest path to a genuinely lightweight bundle.

This deployment reading is also visible in bundle construction. In the current implementation, M2 and the later M6–M10 refinements all consume the same lightweight geometry-plus-street-ref bundle family, whereas M3 reintroduces the `way_links` gate and M4–M5 depend on the corridor tables. A dedicated M2 artifact therefore can omit `corridor_progress`, `corridor_pairs`, and `way_links`. Because the broader replay sweep shows only marginal aggregate differences between M2 and the richer-data variants, the current evidence does not justify carrying `way_links` in the lightweight branch. A later stripped-bundle ablation isolates that side table directly on the current Karlsruhe runtime artifact: removing only `way_links` reduces the raw SQLite bundle from 129.8 MiB to 80.7 MiB and the compressed payload from 47.2 MiB to 33.2 MiB. On the expanded 17-log replay corpus, the same ablation changes only 15 of 26,772 M2 selections, leaves replay accuracy, changed-example recall, and tunnel-fix count unchanged, and shifts logged agreement only from 87.46% to 87.44%. Later street-ref guard variants show the same aggregate pattern: removing `way_links` leaves replay accuracy unchanged and mostly removes only the node-aware distinction between closely related lightweight variants. The existing Karlsruhe benchmark build family already shows the direction of a fully dedicated lightweight artifact: the no-`way_links` geometry variant is 77,869,056 bytes, compared with 107,929,600 bytes for the corresponding way-links variant.

8 Discussion

The results support a simpler storyline than earlier drafts. The architecture and the matcher are no longer one blended question. The platform story is mostly settled: S3 is the best current data layer, the bundle contract is implemented across both clients, and iterative updates have moved from design intent to measured subsystem. That is the strongest conclusion the report can make.

A second result is explanatory rather than competitive: jurisdiction context is not peripheral metadata. Germany's provenance split shows why area context and fallback handling are operationally necessary, and the Europe-wide explicit-tag comparison shows why the rollout portfolio cannot be chosen by file size alone. At the same time, this layer remains asymmetric. Warning semantics already span more countries than the legal-default inference layer, whose strongest validation is still German.

The matcher evidence is now more convergent than earlier drafts suggested. The all-log ladder is still useful because it shows where added sequence machinery helps and where it regresses, but the expanded replay sweep is the stronger deployment test. Under that test, M2 already sits in the same practical quality band as the richer-data street-ref variants, the node-aware endpoint M10 is effectively indistinguishable from M9, and the corridor-heavy branches do not justify their additional data structures on the broader corpus. The stripped-bundle rerun strengthens that reading further: removing only `way_links` leaves

M2's primary replay metrics unchanged while materially reducing bundle size.

The paper's main contribution should therefore be read as a deployment study with one stable answer and one open frontier. The stable answer is the local-first bundle and update architecture centered on S3. The open frontier is how far the consumer matcher can be simplified without surrendering too much replay quality or reintroducing route instability. That separation is what makes the current evidence coherent.

9 Future Work

Several next steps follow directly from the current evidence.

The first concerns jurisdiction-aware defaults. The present bundle contract and warning layer already scale across jurisdictions, but the legal-default inference layer still deserves a fuller country-by-country model. Extending the explicit or inherited or fallback provenance decomposition beyond Germany is therefore not just a reporting improvement; it is also the measurement foundation for more principled country-specific default handling.

The second concerns iterative updates. The exact delta-pack machinery is implemented and measured, but the rollout path remains intentionally conservative. A natural next step is to move from the Karlsruhe seed-first validation pattern to a broader country and shard matrix, with stronger end-to-end evidence for multipart downloads, stale-version recovery, and repeated patch application on real devices rather than only host-side simulation and narrow mobile-path verification.

The third concerns matcher convergence. The current results now resolve four concrete hypotheses. First, directly truncating the corridor-aware runtime at the raw mini-HMM step is not sufficient. Second, the much simpler nearest plus street-ref continuity rule is no longer just a lightweight fallback; on the expanded replay corpus it already operates in the same practical quality band as the richer-data variants. Third, reintroducing the connected-candidate gate into that branch does not help. Fourth, the remaining gains from later lightweight refinements are small enough that bundle and implementation cost now dominate. Future work should therefore proceed on two branches: productize a dedicated M2 artifact family that removes corridor tables and, on present replay evidence, can likely also drop `way_links` without materially changing replay behavior; and test narrow tunnel-specific interventions on top of M2 for the remaining missed-portal cases rather than returning to the corridor-heavy branch. Those experiments should still be scored jointly with bundle build cost, delta size, update friction, and geometry-stress stability so that matcher adoption reflects the full operational envelope of the product.

10 Conclusion

The current YouSpeed report supports one strong systems conclusion and one narrower model-selection conclusion. The strong systems conclusion is that the project now has a coherent offline-first deployment stack: regional bundle publication, coverage-aware routing, on-device lookup, country-specific warning rules, and measured delta-style updates all sit on top of an S3 single-file runtime that remains the best present data-layer choice. This is the stable part of the story.

The narrower model-selection conclusion is also no longer that the richest matcher wins and M2 is merely a lighter fallback. The expanded replay sweep shows that M2 already provides practically the same aggregate quality as the richer-data variants. The best later street-ref models improve replay accuracy by only 0.30 percentage points, M10 is effectively identical to M9, M3 does not justify reintroducing `way_links`, and direct stripped-bundle reruns leave M2's primary replay metrics unchanged. In other words, the architecture is settled and the remaining matcher work is now about targeted lightweight refinements, not about carrying more side-table structure by default.

That is the main value of the current report. It turns an earlier mixed narrative into a structured state-of-the-system account: what is already implemented and stable, why jurisdiction and update mechanics matter, and where the remaining design freedom actually lies.

Reproducibility

All preprocessing, scenario construction, benchmark steps, bundle-generation stages, and update scripts are parameterized in the accompanying codebase at https://github.com/volzinnovation/youspeed.de. The current report ties its measurements to the Germany snapshot dated 2026-02-23, the Europe-wide explicit-tag analysis artifacts stored under `mapdata/reports/`, the 30-day Germany daily-diff analysis window from 2026-01-22 to 2026-02-20, the fixed Berlin probe benchmark, and the replay corpora summarized in Table 14.

A Bundle Artifact Contracts

The runtime artifact family uses three JSON contracts: the bundle manifest, the delta manifest, and the country catalog. Table 20 summarizes the bundle manifest fields. Table 21 summarizes the delta manifest fields. Table 22 summarizes the country catalog fields.

Table 20: Current v3 bundle-manifest fields.

Field	Required	Meaning
`format`	yes	Constant identifier `youspeed.v3.bundle.manifest`.
`schema_version`	yes	Manifest schema version.
`variant`	yes	Runtime family identifier, currently `v3`.
`region`	yes	Logical bundle region id.
`country_code`	no	ISO alpha-3 jurisdiction code used for bundle identity and rule selection.
`bundle_version`	yes	Logical bundle version string.
`created_at_utc`	yes	Manifest creation timestamp.
`min_app_version`	yes	Minimum compatible app version.
`db`	yes	Logical database artifact with file name, bytes, checksum, and optional URL.
`db_parts`	no	Optional list of split database parts for release-asset size limits.
`delta_index`	no	Optional rolling index of available incremental updates.
`penalty_rules`	no	Optional country-specific penalty-rule artifact.
`coverage`	no	Optional bbox plus optional polygon artifact for on-device bundle routing.
`self`	no	Manifest file identity and optional published URL.

Table 21: Current v3 delta-manifest fields.

Field	Required	Meaning
`format`	yes	Constant identifier `youspeed.v3.delta.manifest`.
`schema_version`	yes	Delta schema version.
`region`	yes	Logical region id covered by the patch.
`from_bundle_version`	yes	Source bundle version for the patch.
`to_bundle_version`	yes	Target bundle version for the patch.
`created_at_utc`	yes	Delta-manifest creation timestamp.
`source_diff_file`	no	Input replication diff used to generate the patch.
`patch`	yes	SQL patch artifact with file, bytes, checksum, URL, and compression metadata.
`stats`	yes	Patch statistics including changed ways, speed-tag changes, and SQL row counts.

Table 22: Current v3 country-catalog fields.

Field	Required	Meaning
`format`	yes	Constant identifier `youspeed.v3.bundle.catalog`.
`schema_version`	yes	Catalog schema version.
`variant`	yes	Runtime family identifier, currently `v3`.
`country`	yes	Country identifier for the catalog owner.
`bundle_version`	yes	Logical catalog version.
`created_at_utc`	yes	Catalog creation timestamp.
`max_country_pbf_bytes`	yes	Planning threshold that explains why sharding is or is not used.
`regions`	yes	List of region entries with manifest artifact and optional coverage metadata.

B Current v3 Bundle Database Schema

Table 23 documents the schema of the current corridor-aware Karlsruhe v3 bundle (`bundle_version 2026-03-11-shared-node-corridor-hmm`). Internal RTree shadow tables are omitted for readability. The current builder can also materialize additional intermediate tables such as `way_endpoints` during construction, but the deployed bundle documented here retains the tables that the runtime actually consumes.

Table 23: Current deployed v3 bundle schema and row counts (Karlsruhe latest corridor-aware bundle).

Table	Columns	Purpose	Rows
`metadata`	`key`, `value`	Build metadata, feature flags, source lineage.	8
`ways`	`way_id`, class, name or ref, explicit or inherited speed fields, heading, service, tunnel, bbox	Per-way speed, class, heading, and bounding-box attributes.	242,359
`ways_rtree`	`way_id` plus bbox coordinates	Spatial pruning for candidate way retrieval.	242,359
`way_geom`	`way_id`, `points_json`	Polyline geometry for point-to-segment refinement.	242,359
`areas`	`row_id`, area id, geometry or place fields, residential flag, geometry JSON, bbox	Residential polygons plus place or boundary context.	32,872
`areas_rtree`	`row_id` plus bbox coordinates	Spatial pruning for polygon and area-context lookup.	32,872
`way_links`	`way_id`, linked way id, shared ref, shared node key	Detailed shared-node continuity graph.	435,372
`corridor_progress`	Corridor kind or id, side node key, way id, depth and span measures	Stateful tunnel and motorway corridor arbitration.	11,961
`corridor_pairs`	Corridor id plus paired corridor identity fields	Pair entry and exit corridor components.	1,656

The current `way_links` table uses the following SQL column declarations:

```
way_id INTEGER NOT NULL
linked_way_id INTEGER NOT NULL
shared_ref INTEGER NOT NULL DEFAULT 0
shared_node_key TEXT NOT NULL
```

Its composite primary key spans `way_id`, `linked_way_id`, and `shared_node_key`, and the auxiliary index is `idx_way_links_way_id` on `way_id`. This detail matters for the lightweight-matcher ablation above because the replay reruns remove precisely this side table while leaving the rest of the database unchanged.

C Europe-wide Explicit Maxspeed Ranking

Table 24 reproduces the full current Europe-wide ranking used by the comparative provenance scan. The statistic measures explicit `maxspeed=*` coverage on car-drivable ways and is included here in full because it directly informs target-country prioritization and cross-country provenance interpretation.

Table 24: Europe-wide ranking by explicit `maxspeed=*` share on car-drivable ways.

Rank	Country	ISO2	Drivable ways	Tagged ways	Share (%)
1	Netherlands	NL	1,360,101	921,426	67.75
2	Romania	RO	671,248	419,884	62.55
3	Luxembourg	LU	57,401	30,766	53.60
4	Belgium	BE	816,110	302,452	37.06
5	Sweden	SE	1,235,544	443,443	35.89
6	Liechtenstein	LI	5,407	1,836	33.96
7	Iceland	IS	61,433	20,751	33.78
8	Germany	DE	7,964,480	2,637,436	33.12
9	Monaco	MC	1,224	355	29.00
10	United Kingdom	GB	4,871,056	1,275,216	26.18
11	Andorra	AD	3,714	955	25.71
12	Switzerland	CH	862,842	218,757	25.35
13	Hungary	HU	514,402	126,714	24.63
14	Austria	AT	1,257,719	287,600	22.87
15	Czech Republic	CZ	860,607	187,689	21.81

Continued on next page

Rank	Country	ISO2	Drivable ways	Tagged ways	Share (%)
16	France	FR	7,111,322	1,490,044	20.95
17	Serbia	RS	381,641	77,588	20.33
18	Slovakia	SK	367,681	74,103	20.15
19	Malta	MT	26,092	4,658	17.85
20	Ireland and Northern Ireland	IE	1,096,604	195,628	17.84
21	Denmark	DK	886,408	157,710	17.79
22	Finland	FI	996,944	167,987	16.85
23	Spain	ES	2,720,041	439,358	16.15
24	Belarus	BY	534,645	85,966	16.08
25	Croatia	HR	298,960	47,342	15.84
26	Norway	NO	1,207,883	189,060	15.65
27	Poland	PL	3,030,384	467,033	15.41
28	Lithuania	LT	327,952	49,525	15.10
29	Estonia	EE	182,364	26,243	14.39
30	Cyprus	CY	115,415	16,038	13.90
31	Bulgaria	BG	368,235	49,724	13.50
32	Italy	IT	4,005,392	539,504	13.47
33	Montenegro	ME	56,044	7,498	13.38
34	Macedonia	MK	68,219	8,943	13.11
35	Latvia	LV	263,163	32,967	12.53
36	Faroe Islands	FO	13,226	1,654	12.51
37	Slovenia	SI	252,487	29,961	11.87
38	Greece	GR	888,354	104,360	11.75
39	Portugal	PT	870,579	102,083	11.73
40	Albania	AL	125,492	11,242	8.96
41	Ukraine (with Crimea)	UA	1,441,203	114,036	7.91
42	Georgia	GE	189,701	11,521	6.07
43	Bosnia-Herzegovina	BA	214,701	10,729	5.00
44	Moldova	MD	188,319	9,175	4.87
45	Turkey	TR	1,902,558	59,063	3.10

References

[1] European Parliament and Council of the European Union. Regulation (eu) 2019/2144 on type-approval requirements for motor vehicles and their trailers, and systems, components and separate technical units intended for such vehicles. https://eur-lex.europa.eu/eli/reg/2019/2144/oj, 2019. Accessed: 2026-02-23.

[2] European Commission. Commission delegated regulation (eu) 2021/1958 supplementing regulation (eu) 2019/2144 by laying down detailed rules concerning intelligent speed assistance and driver drowsiness and attention warning systems. https://eur-lex.europa.eu/eli/reg_del/2021/1958/oj, 2021. Accessed: 2026-02-23.

[3] European Commission. New mandatory vehicle safety features from july 2024. https://single-market-economy.ec.europa.eu/news/new-mandatory-vehicle-safety-features-2024-07-05_en, 2024. Accessed: 2026-02-23.

[4] Eurostat. Passenger cars in the eu (statistics explained; data extracted in january 2026). https://ec.europa.eu/eurostat/statistics-explained/SEPDF/cache/25886.pdf, 2026. Accessed: 2026-02-23.

[5] European Automobile Manufacturers' Association (ACEA). Car registrations: +0.8% in 2024; battery electric 13.6% market share in december. https://www.acea.auto/pc-registrations/new-car-registrations-0-8-in-2024-battery-electric-13-6-market-share-in-december/, 2025. Accessed: 2026-02-23.

[6] Mohammed A. Quddus, Washington Y. Ochieng, and Robert B. Noland. Current map-matching algorithms for transport applications: State-of-the art and future research directions. *Transportation Research Part C: Emerging Technologies*, 15(5):312–328, 2007. doi: 10.1016/j.trc.2007.05.002. URL https://doi.org/10.1016/j.trc.2007.05.002.

[7] Paul Newson and John Krumm. Hidden markov map matching through noise and sparseness. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 336–343, 2009. doi: 10.1145/1653771.1653818. URL https://doi.org/10.1145/1653771.1653818.

[8] Yin Lou, Chengyang Zhang, Yu Zheng, Xing Xie, Wei Wang, and Yan Huang. Map-matching for low-sampling-rate gps trajectories. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 352–361, 2009. doi: 10.1145/1653771.1653820. URL https://doi.org/10.1145/1653771.1653820.

[9] C. Y. Goh, J. Dauwels, N. Mitrovic, M. T. Asif, A. Oran, and P. Jaillet. Online map-matching based on hidden markov model for real-time traffic sensing applications. In *2012 15th International IEEE Conference on Intelligent Transportation Systems*, pages 776–781, 2012. doi: 10.1109/ITSC.2012.6338627. URL https://doi.org/10.1109/ITSC.2012.6338627.

[10] Zhao-cheng He, She Xi-wei, Li-jian Zhuang, and Pei-lin Nie. On-line map-matching framework for floating car data with low sampling rate in urban road networks. *IET Intelligent Transport Systems*, 7 (4):404–414, 2013. doi: 10.1049/iet-its.2011.0226. URL https://doi.org/10.1049/iet-its.2011.0226.

[11] Timothy Hunter, Pieter Abbeel, and Alexandre Bayen. The path inference filter: Model-based low-latency map matching of probe vehicle data. *IEEE Transactions on Intelligent Transportation Systems*, 15(2):507–529, 2014. doi: 10.1109/TITS.2013.2282352. URL https://doi.org/10.1109/TITS.2013.2282352.

[12] Can Yang and Gyözö Gidófalvi. Fast map matching, an algorithm integrating hidden markov model with precomputation. *International Journal of Geographical Information Science*, 32(3):547–570, 2018. doi: 10.1080/13658816.2018.1474007. URL https://doi.org/10.1080/13658816.2018.1474007.

[13] Michel Bierlaire, Jingmin Chen, and Jeffrey Newman. A probabilistic map matching method for smartphone gps data. *Transportation Research Part C: Emerging Technologies*, 26:78–98, 2013. doi: 10.1016/j.trc.2012.08.001. URL https://doi.org/10.1016/j.trc.2012.08.001.

[14] Project OSRM Contributors. Osrm api documentation: Match service. https://project-osrm.org/docs/v5.24.0/api/#match-service, 2026. Accessed: 2026-02-26.

[15] Valhalla Contributors. Valhalla meili documentation. https://valhalla.github.io/valhalla/meili/, 2026. Accessed: 2026-02-26.

[16] Mapbox. Map matching api. https://docs.mapbox.com/api/navigation/map-matching/, 2026. Accessed: 2026-02-26.

[17] Zhixiong Jin, Jiwon Kim, Hwasoo Yeo, and Seongjin Choi. Transformer-based map-matching model with limited labeled data using transfer-learning approach. *Transportation Research Part C: Emerging Technologies*, 140:103668, 2022. doi: 10.1016/j.trc.2022.103668. URL https://doi.org/10.1016/j.trc.2022.103668.

[18] Reza Safarzadeh and Xin Wang. Map matching on low sampling rate trajectories through deep inverse reinforcement learning and multi-intention modeling. *International Journal of Geographical Information Science*, 38(12):2648–2683, 2024. doi: 10.1080/13658816.2024.2391411. URL https://doi.org/10.1080/13658816.2024.2391411.

[19] Yingxue Zhang, Haowen Yan, and Xiaomin Lu. An enhanced hmm map matching algorithm incorporating personal road selection preferences. *Scientific Reports*, 15:35962, 2025. doi: 10.1038/s41598-025-14050-8. URL https://doi.org/10.1038/s41598-025-14050-8.

[20] Fei Meng and Jiale Zhao. A robust meta-learning-based map-matching method for vehicle navigation in complex environments. *Symmetry*, 18(1):210, 2026. doi: 10.3390/sym18010210. URL https://doi.org/10.3390/sym18010210.

[21] Mapbox. Mbtiles specification. https://github.com/mapbox/mbtiles-spec, 2026. Accessed: 2026-03-13.

[22] Protomaps. Pmtiles concepts and specification overview. https://docs.protomaps.com/pmtiles/, 2026. Accessed: 2026-03-13.

[23] OpenStreetMap Wiki Contributors. Key:highway. https://wiki.openstreetmap.org/wiki/Key:highway, 2026. Accessed: 2026-02-23.

[24] OpenStreetMap Wiki Contributors. Key:maxspeed. https://wiki.openstreetmap.org/wiki/Key:maxspeed, 2026. Accessed: 2026-02-23.

[25] Geofabrik GmbH. Geofabrik openstreetmap data extracts – germany latest. https://download.geofabrik.de/europe/germany-latest.osm.pbf, 2026. Accessed: 2026-02-23.